Promo Finder: curiosity + rank fusion for microdrama promos

Problem 1: Find the Promos. Prajyot Rajesh Bhagat · 23123012 · Engineering Physics ·
Prototype: link

The idea

A promo works when a stranger **needs to know what happens next**. Psychology describes curiosity as the discomfort of an **information gap**, a question you feel but can't yet answer (Loewenstein, 1994). The tool therefore tracks every question the story raises and answers, and looks for clips that **cut while a question is still open**.

It doesn't trust any single scorer. Several cheap, independent engines each rank the clips, and their rankings are **fused** with Reciprocal Rank Fusion (Cormack et al., SIGIR 2009), which combines ranks rather than scores. On my labelled test it matches my earlier versions' accuracy at about half the token cost.

Assumptions:

- Script time equals video time.
- The target audience is a US stranger viewing the content on TikTok/Reels.
- The promo links directly to Episode 1.
- A "hook" means something happens within the first 0–3 seconds.
- Jump cuts and flash-forwards are allowed.
- The evaluation is based on the script only; acting and performance are not visible.
- The editor makes the final decision.

How it works

Stage	Cost	What it does	Grounded in
Rules	free	Scores hook, cliffhanger, intensity and standalone-ness of every clean 20–60s window	—
Annotate	10 small-model calls, cached per episode	Labels 2,037 beats: arousal, surprise, context need, turning point, **questions opened / answered** (272 tracked)	Turning points + sentiment drive trailer selection (Papalampidi, Keller & Lapata 2019, 2021)

Stage	Cost	What it does	Grounded in
Curiosity optimizer	free	Searches every window and *flash-forward cold open*; rewards the **gap left open at the cut**, penalizes needed backstory and self-resolution	Loewenstein 1994; suspense as uncertainty (Wilmot & Keller, ACL 2020)
LLM picker	10 small-model calls, compact prompt	A second, independent generator; names beat ids and code owns timestamps	—
RRF fusion	free	score = Σ 1/(60 + rank) over rankers; clips covering ≥50% of each other count as one **moment**, so a moment found by several engines collects all their votes	Cormack, Clarke & Büttcher, SIGIR 2009
Optional: tournament + persona panel	+4 stronger-model + 5 calls	Pairs judged in both orders with Bradley-Terry; five personas predict the second they'd swipe	Zheng et al. 2023; Argyle et al. 2023

A validator never cuts mid-line or mid-sentence and enforces 20–60s. Changing the model is one flag (`gemini:`, `openai:`, `anthropic:`, `hf:`, `groq:`, `ollama:`, or any OpenAI-compatible URL). `--budget B0…B4` picks how much to spend. An **input check** flags broken or templated scripts before any call is made.

What an editor gets: a web UI (`streamlit run app.py`) with the top 10 promos (hook, cliffhanger, the questions left open, which engines found it), a script view and a curiosity chart per clip, an episode explorer, 👍/👎 review saved as feedback, and each promo

exported as **its own script** in the episode format, timed from 00:00 (all 10 round-trip through the parser exactly). Also `report.md`, `promos.csv` and `promos.json`.

Results (all 50 episodes)

Test: is the top-ranked clip in an episode the moment I hand-labelled? Dev means 8 episodes used while building. **Held-out** means 6 episodes labelled later, *before* looking at any picks there, and never used for tuning. The fusion method (RRF, k=60, equal weights) was fixed before scoring the held-out set.

Engine	LLM calls / input tokens	Dev	**Held-out**	Total
Random 20–60s clips	0	4.4/8	2.4/6	6.8
v1: LLM picks directly	13 / ~230k	8/8	4/6	12
v2: optimizer + picks → tournament → panel	29 (4 on stronger model) / ~378k	7/8	5/6	12
v3 B0: rules only	0	6/8	5/6	11
v3 B1: + annotation + optimizer, fused	10 / ~104k	6/8	5/6	11
v3 B2: + compact LLM picker, fused (default)	**20 / ~213k**	**8/8**	**5/6**	**13**

Cost techniques, each measured:

1. **Fusion instead of bigger models:** adding the stronger-model tournament and panel on top of B2 did not change accuracy (13/14 either way), so the default drops them: 31% fewer calls and 44% fewer input tokens than v2.
2. **Compact prompt:** the picker skips plain camera directions, so its prompt is 47% shorter (221k → 109k tokens) with no accuracy loss.
3. **Content-addressed cache (files or Redis):** keyed by a hash of model + instructions + episode text, so a weekly drop of 5 episodes costs 2 calls (about 10% of a full run) and re-runs are free.
4. **Ranks beat averaging:** RRF 13/14, Borda 12/14, z-score averaging 9/14.

Other findings:

- **Pairwise judging is reliable:** 83% of pairs got the same winner in both orders.
- **Generators are complementary:** LLM picks beat optimizer clips 60/40 head-to-head, but the optimizer finds moments the LLM never proposes. The #1 pick was found independently by three engines.

Post first:

1. **Ep 11:** a search turns into a frame-up, cutting on "There it is. Ha!"
2. **Ep 31:** Catherine taunts the chained Juliana and the hot iron stops inches from her eyes.
3. **Ep 16** (the tool's #4): the thugs' "Stay back!", cutting on Juliana realising "Gerald…". I'd skip the tool's #3 (Ep 33), which ends on a maid quietly weeping, a soft ending.

Where it picks badly

- **Tiny evaluation.** 15 moments, one labeller (me). Differences of one episode are noise (batching alone moved one model by ±3), so the safe claims are "same accuracy for about half the cost" and "beats random clearly".
- **Not compared:** other LLMs (only Gemini was available; Hugging Face models were blocked by missing credits), commercial clip tools (they need video), human editors, or real ad results.
- **Fusion favors safe consensus.** Moments everyone agrees on win, and some end softly (Ep 33 ends on a girl quietly weeping).
- **Coarse annotations.** Flash-lite labels many beats as 1, and some episodes get only about 3 questions.
- **Unchecked context.** In B2, nobody checks whether a cold viewer gets the context. The optional panel caught "why run into a fire *for a book*?" (Ep 18). Use B4 for the clips you actually spend ad money on.

Automation and scale

- **Agent** (`python -m promo_finder.agent`, from cron): plans the exact calls from the caches, stays within a daily budget, runs v3 and writes a digest of top-10 changes. Measured: new show 20 calls, weekly drop of 5 episodes 2, unchanged show 0.
- **Second, unseen show** (49 episodes, different timestamp format): exactly 20 calls in 90 s. The input check flagged it **templated** before spending anything (83% of dialogue repeats in every episode); repeated footage is kept out of the top 10.
- **Scale** (`docs/SCALING.md`): the provider's rate limit is the bottleneck, not CPU (12.6 s, 47 MB per show). Workers share one Redis (cache, per-show locks, daily budget, requests-per-minute limit); 2 workers on 4 shows split them with no double work. 500 new + 3,000 running shows a week ≈ 16k calls, about 16 min of API time at 1,000 RPM.
- **Next:** learn the fusion weights from editor 👍/👎 and real hold/completion/installs, then live A/B tests, then video signals.

AI tools used and where they got it wrong

Claude Code built the code with me directing and checking it. **Gemini 3.5 Flash-Lite and Flash** are the models inside the tool.

Where they got it wrong: Gemini's JSON mode wrapped plain-text labels in JSON (0 of 182 beats parsed until I caught it); flash-lite returned 2-minute "promos"; the first fix for long clips trimmed away the hook; the persona panel first rated everything 80–100% completion.

References: Loewenstein (1994) *Psych. Bulletin*; Papalampidi, Keller & Lapata (2019) *EMNLP*, (2021) arXiv:2111.08774; Wilmot & Keller (2020) *ACL*; Cormack, Clarke & Büttcher (2009) *SIGIR*; Zheng et al. (2023) *NeurIPS*; Argyle et al. (2023) *Political Analysis*.